



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2333 — Sistemas Operativos y Redes — 1/2026

Tarea 1 - *Scheduling*

Lunes 6 de abril 2026

Fecha de Entrega: Lunes 27 de abril a las 22:00

Composición: Tarea en parejas

Objetivos

- Modelar un *scheduler* de procesos utilizando `structs`.
- Simular la ejecución de dicho *scheduler* sobre un grupo de procesos.

DCCyclone



Figura 1: Homero Simpson

Todo comenzó durante una jornada de trabajo rutinaria en la consola de control principal. Homero Simpson, reclinó su silla y colocó sus pies sobre el tablero de mandos. Al hacerlo, presionó un botón crítico que desactivó el *firewall* del sistema central.

Aprovechando esta brecha de seguridad, ingresó mediante la red un virus que funciona como un “*Cyclone*” informático que desordenó instantáneamente todos los procesos de la computadora central. Los protocolos de enfriamiento, los sensores de presión y los sistemas de emergencia han quedado revueltos en las colas de ejecución.

El planificador original ha colapsado y, si los procesos críticos no logran ejecutarse según su prioridad por *deadline*, el sistema entrará en una falla masiva que provocará el caos total y la inminente explosión de la planta nuclear.

Ante la emergencia, se te ha pedido urgentemente iniciar el protocolo **DDCyclone**: El cual tiene como objetivo crear un *scheduler* eficiente diseñado para reorganizar el sistema y asegurar que cada proceso finalice antes de su *deadline*.

Descripción

El *scheduling* consiste en elegir el siguiente proceso a ejecutar en la CPU. Dado que no siempre se tiene información completa sobre los procesos, existen distintos algoritmos que buscan tomar decisiones eficientes en función de ciertos criterios, como prioridades, tiempos de ejecución o fechas límite.

En esta tarea se propone el diseño e implementación de un scheduler MLFQ que combina distintos enfoques para manejar procesos con restricciones de tiempo. En particular, el sistema utiliza el algoritmo *Earliest Deadline First* (EDF) para priorizar procesos según su deadline, junto con una política de *Round Robin* (RR) para asegurar equidad en el uso de la CPU.

El scheduler está compuesto por dos colas: una de alta prioridad (*HIGH*) y otra de baja prioridad (*LOW*). La cola *HIGH* contiene procesos críticos o interactivos y opera de manera preemptiva utilizando EDF, complementado con un esquema de quantum que evita la monopolización de la CPU. Por otro lado, la cola *LOW* agrupa procesos más intensivos en CPU, los cuales se ejecutan con menor prioridad.

Para evitar la inanición de procesos en la cola *LOW*, se incorpora un mecanismo de *aging*, donde aquellos procesos que permanecen demasiado tiempo en dicha cola son promovidos nuevamente a *HIGH*. Este criterio permite garantizar que todos los procesos eventualmente reciban atención, incluso en presencia de cargas de trabajo altamente demandantes.

Modelamiento de procesos

Debe construir un `struct Process`, que modelará un proceso. Cada proceso tiene al menos las siguientes variables asociadas:

1. Nombre.
2. PID.
3. Estado $\in \{\text{RUNNING}, \text{READY}, \text{WAITING}, \text{FINISHED}\}$.
4. Tiempo de ejecución por ráfaga (*burst*).
5. Número de ráfagas de ejecución en CPU.
6. Tiempo de espera para I/O entre ráfagas.
7. Deadline de ejecución.

Modelamiento de las colas de procesos

También debe existir un `struct Queue`, el cual modelará una cola de procesos. Esta estructura debe estar diseñada por ustedes y debe ser capaz de recibir un número arbitrario de procesos en estado *READY* de manera simultánea.

Cada cola mantiene su propia política de scheduling:

- **Cola *High*:** se implementa como EDF + RR con quantum. El parámetro global q define el quantum de *High*: $quantum_{High} = q$.
- **Cola *Low*:** se implementa como EDF sin quantum (solo prioridad por deadline). Los procesos en *Low* no están sujetos a reducción de quantum, se mantienen ahí hasta que un evento de *aging* o el proceso entre en estado *WAITING* por I/O, y deban moverse a *High*.

(En todos los casos se respeta que la cola *High* tiene prioridad absoluta sobre la cola *Low*).

Scheduler MLFQ con EDF + Aging

El *scheduler* puede contener procesos en las colas. Los procesos en una cola pueden estar en dos estados posibles:

- **READY**, indica que el proceso está listo para ejecutar.
- **WAITING**, indica que el proceso está esperando en I/O y no puede ejecutar.

En caso de que un proceso no se encuentre dentro de una cola, sus estados pueden ser:

- **RUNNING**, indica que el proceso está ejecutando dentro de la CPU. Sólo puede haber un proceso en este estado a la vez.
- **FINISHED**, indica que el proceso ha completado toda su ejecución.
- **DEAD**, indica que el proceso no cumplió su *deadline*.

El scheduler debe asegurar que el estado de cada proceso cambie de manera consistente.

Scheduler MLFQ

Su programa debe implementar un *scheduler* de dos colas, *High* y *Low*, con ordenamiento por *Earliest Deadline First* (EDF) dentro de cada cola, de acuerdo a las siguientes reglas:

- Los procesos de la cola *High* siempre tienen prioridad por sobre la cola *Low*.
- Todo proceso que ingresa al *scheduler* por primera vez, o que completa una operación de I/O, lo hace en la cola *High* y en estado **READY** (I/O boost).
- Si dos o más procesos tienen la misma prioridad (están en la misma cola), estos deberán ser ejecutados según ordenamiento EDF:
 - Los procesos con mayor prioridad dentro de una misma cola serán aquellos con menor valor de $T_{deadline}$ (más cercano a cumplir su vencimiento).
 - Si hay empate en *deadline*, se elige el que tenga menor PID.
 - Los procesos en estado **WAITING** no son elegibles para ejecutar.
- Un proceso en estado **RUNNING** de la cola *High* puede ser interrumpido si llega un proceso con *deadline* anterior al actual.
- Si un proceso consume todo su quantum, este pasa a la cola *Low*. El quantum se acumula, durante la ejecución de cada proceso se va consumiendo su quantum, y se reinicia solo al cambiar de cola.
- Si un proceso termina una ráfaga de CPU, este pasa a estado **WAITING** para realizar I/O.
- Si se cumple para un proceso en la cola *Low* que $(T_{actual} - T_{LCPU}) > Threshold_S^1$, este proceso asciende a la cola *High*.
- Si un proceso alcanza su *deadline* ($T_{actual} \geq T_{deadline}$) y no ha terminado, cambia a estado **DEAD**.

¹ Donde T_{LCPU} es el tiempo en que el proceso salió por última vez de la CPU

Flujo del *scheduler*

Por cada *tick*, el *scheduler* debe realizar las siguientes tareas, en el orden indicado:

1. **Llegadas:** Ingresar los procesos cuyo tiempo de inicio es el tick actual en la cola *High*.
2. **Retorno de I/O:** Actualizar procesos en estado `WAITING` que han completado su tiempo de I/O a estado `READY`.
3. **Ascenso:** Para cada proceso en la cola *Low*, revisar si $(T_{actual} - T_{LCPU}) > Threshold_S$ y ascenderlo a la cola *High* si corresponde.
4. **Verificación de deadlines:** Marcar como `DEAD` todo proceso en estado `READY` o `WAITING` que ya haya pasado su *deadline*.
5. **Ordenamiento:** Re-ordenar ambas colas por deadline (EDF).
6. **Ejecución:** Incrementar los contadores del proceso en ejecución:
 - 6.1) Si termina su último burst y no hay ráfaga restante: cambiar a `FINISHED`.
 - 6.2) Si llega a su deadline absoluto: cambiar a `DEAD`.
 - 6.3) Si termina una ráfaga (pero no es la última): cambiar a `WAITING` para I/O.
 - 6.4) Si consume todo su quantum: cambiar a `READY` y descender a la cola *Low* si estaba en *High*.
 - 6.5) Si hay un proceso con mayor prioridad en estado `READY`, pasar proceso en ejecución a estado `READY` y regresarlo a su cola de origen.
7. **Dispatch:** Si no hay proceso en `RUNNING`, seleccionar el proceso de mayor prioridad en estado `READY` de la cola *High*. Si está vacía, hacerlo de la cola *Low*.

Consideraciones especiales

- Si un proceso termina su ráfaga de CPU al mismo momento que se acaba su quantum, se considera que pasa a I/O (prioridad a fin de ráfaga).
- Recordar que contamos con I/O Boost por lo que cualquier proceso que vuelve de I/O regresa a la cola *High*
- Un proceso solo resetea su quantum si es que vuelve de la cola *Low*, es decir, si realiza I/O en cola *High* este no resetea su quantum
- Cuando un proceso se le acaba su quantum o es interrumpido por prioridad, esta igualmente alcanzara a ejecutar un último `tick` antes de dejar la CPU.
- Si un proceso termina su ejecución completa al mismo momento que se acaba su quantum o su deadline, este pasa a estado `FINISHED` (prioridad a fin de ejecución).
- El quantum es acumulativo: si un proceso es interrumpido o este cede la CPU, mantiene su progreso de quantum consumido y vuelve en estado `READY` en la cola *High*.
- Si un proceso es interrumpido en medio de su ráfaga de burst, el burst pendiente queda guardado para ser retomando en su próxima ejecución.

Ejecución de la simulación

La simulación se debe ejecutar con el siguiente comando:

```
./DCCyclone <input_file> <output_file>
```

Donde `input_file` es el nombre del archivo de *input*, `output_file` corresponde a la ruta a un archivo CSV con las estadísticas que debe guardar el programa. Una posible ejecución sería `./oscheduler procesos.txt salida.csv`.

Archivo de entrada (*input*)

El archivo de entrada contiene los datos de la simulación, donde la primera línea contiene un entero q que indica el quantum para la cola HIGH, luego un entero S que indica el *Threshold* de la cola LOW, después un entero K siendo la cantidad de procesos y las siguientes K líneas tienen el siguiente formato:

NOMBRE_PROCESO	PID	T_INICIO	T_CPU_BURST	N_BURSTS	IO_WAIT	T_DEADLINE
----------------	-----	----------	-------------	----------	---------	------------

Donde:

- PID es el Process ID del proceso.
- T_INICIO es el tiempo en el que el proceso entra a la cola por primera vez.
- T_CPU_BURST es el tiempo que dura una ráfaga de ejecución en la CPU.
- N_BURSTS es la cantidad de ráfagas de ejecución en la CPU que tiene el proceso.
- IO_WAIT es el tiempo de espera entre ráfagas de ejecución.
- T_DEADLINE es el tiempo absoluto del deadline de ejecución.

Finalmente:

- T_DEADLINE al ser el tiempo absoluto, significa que, para cada proceso, llegado al $tick = T_DEADLINE$ el proceso muere.
- Puede suponer que no habrá tiempos negativos.
- Puede suponer que para cada proceso $T_DEADLINE > T_INICIO$.
- También pueden asumir que $T_DEADLINE > 0$ y $IO_WAIT \geq 0$. Todos los tiempos se entregan sin dimensión y caben en un entero sin signo de 32 bits.
- No se entregarán archivos vacíos.

Archivo de salida (*output*)

En su archivo de salida, debe incluir una fila con las siguientes estadísticas para cada proceso:

- Nombre del proceso
- PID
- Estado del proceso
- Número de interrupciones
- El turnaround time

- El response time, considerando el tiempo que tardó en entrar por primera vez a la CPU.
- El waiting time, este es la suma de todos los intervalos en que estuvo en estado WAITING o READY para ejecutar.

El orden en el que se imprimen los procesos es por su id, en orden ascendente. Por lo tanto, el archivo de salida se vería como sigue:

```
nombre_proceso_a,pid_a,estado_a,interrupciones_a,turnaround_a,response_a,waiting_a
nombre_proceso_b,pid_b,estado_b,interrupciones_b,turnaround_b,response_b,waiting_b
nombre_proceso_c,pid_c,estado_c,interrupciones_c,turnaround_c,response_c,waiting_c
```

Es importante que **respeten el formato del output**.

Ejemplo Input y Output

A continuación, se les presenta un input y output válidos.

Input:

```
4
5
3
P0 0 1 2 1 1 10
P1 1 2 3 1 1 15
P2 2 3 1 1 1 20
```

Output:

```
P0,0,FINISHED,0,2,0,0
P1,1,FINISHED,0,4,1,1
P2,2,FINISHED,0,4,3,3
```

Formalidades

A cada alumno se le asignó un nombre de usuario y una contraseña para el servidor del curso². Para entregar su tarea usted deberá crear una carpeta llamada T1 en el directorio principal de su carpeta personal y subir su tarea a esa carpeta. En su carpeta T1 **solo debe incluir el código fuente** necesario para compilar su tarea y un `Makefile`. Se revisará el contenido de dicha carpeta el día Lunes 27 de abril a las 22:00.

- La tarea debe ser realizada solamente en parejas.
- La tarea deberá ser realizada en el lenguaje de programación C. Cualquier tarea escrita en otro lenguaje de programación no será revisada.
- **NO debe incluir archivos binarios**³. En caso contrario, tendrá un descuento de 0.2 décimas en su nota final.
- **NO debe incluir un repositorio de git**⁴. En caso contrario, tendrá un descuento de 0.2 décimas en su nota final.
- **NO debe usar VSCode para entrar al servidor**⁵. En caso contrario, tendrá un descuento de 0,2 puntos en su nota final.
- Si inscribe su grupo de forma incorrecta o no lo inscribe, tendrá un descuento de 0.3 décimas
- Su tarea debe compilarse utilizando el comando `make`, y generar un ejecutable llamado `DDCyclone` en esa misma carpeta. Si su programa **no tiene** un `Makefile`, tendrá un descuento de 0.5 décimas en su nota final y corre riesgo que su tarea no sea corregida.
- En caso de no cumplir con el formato de entrega, tendrá un descuento de 0.3 décimas en la nota final.
- Si ésta **no compila** o **no funciona** (*segmentation fault*), obtendrán la nota mínima, pudiendo recorrer modificar líneas de código con un descuento de una décima por cada cuatro líneas modificada, con un máximo de 20 líneas a modificar.

El no respeto de las formalidades o un código extremadamente desordenado podría originar descuentos adicionales, los cuales quedarán a discreción del ayudante corrector. Se recomienda modularizar, utilizar funciones y ocupar nombres de variables explicativos. En el caso de no entregar en la carpeta especificada la tarea, **no** se corregirá.

Evaluación

- **1.5 pts.** Pasa correctamente los Test *Easy*.
- **1.5 pts.** Pasa correctamente los Test *Medium*.
- **2.5 pts.** Pasa correctamente los Test *High*
- **0.5 pts.** Manejo de memoria. Se obtiene este puntaje si `valgrind` reporta en su código 0 *leaks* y 0 errores de memoria en **todo caso de uso**⁶.

² iic2333.ing.puc.cl

³ Los archivos que resulten del proceso de compilar, como lo son los ejecutables y los *object files*

⁴ Si es que lo hace, puede eliminar la carpeta oculta `.git` antes de la fecha de entrega.

⁵ Si es que lo hace, puede eliminar la carpeta oculta `.vscode-server` antes de la fecha de entrega.

⁶ Es decir, debe reportar 0 *leaks* y 0 errores para todo *test*

Política de atraso

Se puede hacer entrega de la tarea con un máximo de 2 días de atraso. La fórmula a seguir es la siguiente:

$$N_{T_1}^{\text{Atraso}} = \min(N_{T_1}, 7,0 - 0,75 \cdot d)$$

Siendo d la cantidad de días de atraso. Notar que esto equivale a un descuento *soft*, es decir, cambia la nota máxima alcanzable y no se realiza un descuento directo sobre la nota obtenida. El uso de días de atraso no implica días extras para alguna tarea futura, por lo que deben usarse bajo su propio riesgo.

Preguntas

Cualquier duda preguntar a través del [foro oficial](#).